“长沙银行杯”2026中国研究生创芯大赛·EDA精英挑战赛
赛题指南

一、赛题名称

AI驱动的PCB原理图自动解析

二、命题单位

芯粒CAD和制造浙江省工程研究中心，比昂芯科技有限公司

三、赛题背景

传统PCB原理图设计高度依赖人工绘制与整理，流程繁琐、迭代周期长、人为误差率高。当前大量硬件开源资料、设备图纸、芯片参考设计均以PDF、PNG等静态图像格式存储，无法直接编辑复用与工程导入，极大限制了电路设计数字化、自动化落地。

近年来，依托人工智能技术实现原理图智能化解析，已成为PCB设计EDA领域的重要研究方向，整体方案主要包含三类技术体系：

第一，传统机器视觉方案。依托边缘检测、霍夫变换、骨架提取、形态学处理、连通域分析等算法，实现导线、器件轮廓、节点的基础分割与提取，结合模板匹配完成常规标准器件识别。

第二，深度学习检测与分割方案。基于YOLO、Faster R-CNN、Mask R-CNN、U-Net等主流视觉模型，实现元器件目标检测、引脚点位分割、导线区域语义分割，结合CRNN、EasyOCR等文字识别算法完成位号、参数、引脚文本提取，通过距离匹配、IOU匹配、空间方位约束实现器件与文字关联绑定，有效提升复杂图纸的识别鲁棒性。

第三，多模态大模型融合方案。依托视觉大模型、图文理解模型的通用感知能力，结合EDA专业先验知识，实现非标器件、模糊图纸、复杂拓扑、歧义标注的智能理解，弥补传统CV算法泛化性差、规则固化的缺陷，提升复杂真实

场景下的电路解析精度。

在基础电路解析方面工业界和学术界都开展了大量工作，并取得突破。但对复杂PCB原理图识别与解析，行业仍存在显著短板和诸多挑战，技术难点包括下面几个方面。

一是密集器件识别精度不足。现有模型对高密度排布、器件重叠、局部遮挡、小型贴片器件场景识别误差大，极易出现器件漏检、误检、类别混淆问题，难以满足工程级精度要求。

二是引脚定位与关联精度低。传统方案仅能识别器件整体包围盒，无法精准区分多引脚器件的引脚坐标、引脚编号对应关系，普遍存在引脚错位、引脚漏识别、引脚与器件错绑定的问题。

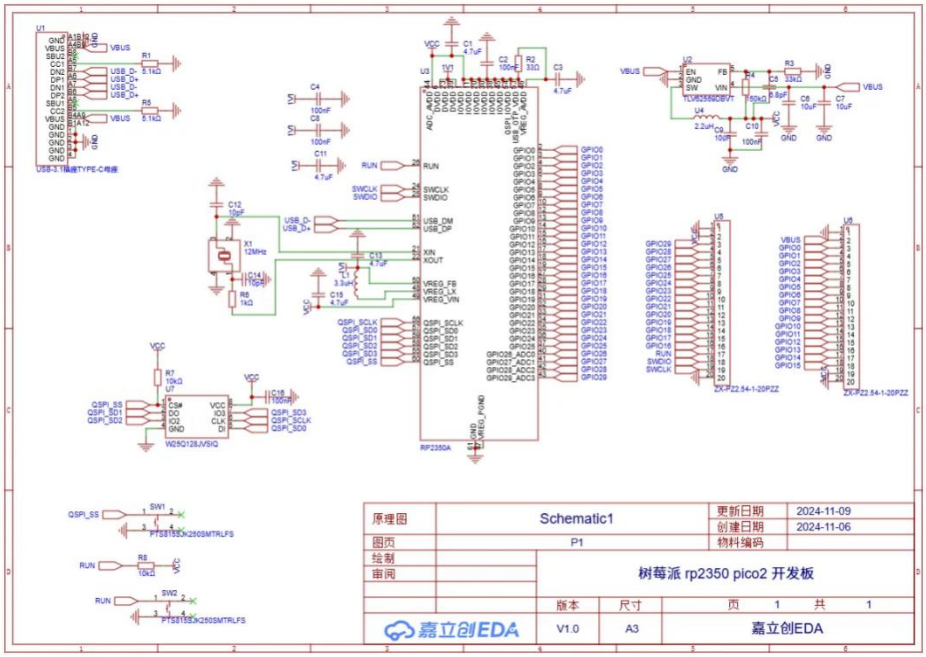
三是文字-器件关联容错性差。常规IOU匹配、就近匹配算法在文字扎堆、标注倾斜、跨器件文字干扰场景下极易错配，无法精准实现位号、参数、引脚名与对应器件的语义绑定。

四是复杂拓扑电气解析能力弱。现有算法难以精准区分导线物理交叉与电气连通，对T型节点、十字节点、多分支走线、断线失真图纸的连通性判断误差大，易出现错连、漏连、虚假网络。

针对前述技术难点和挑战，本赛题聚焦复杂PCB原理图的识别与自动解析技术，为解决这些难点提供创新平台。图一用一个示例说明本赛题需解决问题的特点，包括下面两个方面。

1）赛题需要解析的电路规模大，元器件多，单一器件引脚多并密集，连接关系复杂，文字扎堆易受干扰，飞线抑制难度大。

2）赛题要求完成丰富的信息识别，包括元器件（元件key、类型、名称、value 和bbox），每个元器件上的 pin角（pin number、pin name和pin坐标），电路连接关系（每个net连接了哪些 pins，以及该 net 对应的线段集合）。另外为准确完成本赛题任务，需要完成文字识别，这对复杂的PCB场景有很高的难度。



图一：需识别的PCB原理图例子

（来源：https://oshwhub.com/ 嘉立创开源广场）

四、赛题描述

（一）赛题任务

本赛题任务是给定一张PCB电路原理图图片，参赛系统需要识别出结构化原理图信息，并输出为规定的 JSON 格式。

1.输入：电路原理图图片。赛题提供的数据集为PNG格式，参赛队伍可根据需求转换成其它格式以用于更多处理，赛题对此不做约束。

参赛系统需要预测三类信息：

a)‘components’：图中的元器件实例，例如 `U1`、`R3`、`C12`，包含类别、文本信息和 bbox。

b)‘pins’：每个元器件的 pin，包括pin number、pin name和pin坐标。

c)‘nets’：电路连接关系，包括每个net连接的pins，以及该 net 对应的线段集合。

2. 输出：结构化原理图信息文件，依照赛题规定的JSON格式。附录一结合图一中的案例给出输出JSON格式详细介绍。同时附录二给出一个完整最小输出示例。

3. 赛题提供：a)元器件库用来模糊匹配，精准确定元器件, b）Golden data 数据集，数据集会根据难度进行分类，分为简单，中等，难。参赛队伍可与赛题提供的golden data 进行对比修正，优化模型和算法。

赛题官方数据集为 PCBnet, 数据集信息和网址如下，参考文献[1]。

数据集名称：PCBnet (面向 PCB 原理图理解与网表构建的大规模数据集)

网站：https://github.com/ECNUACRush/PCBnet/

4.参赛队伍可在赛题提供的数据集基础上自行收集数据，用于模型训练和算法优化。

5.最终评分综合考虑：

a)元件是否识别正确。

b)元件 type / text / bbox 是否正确。其中 IC 比较 'Name'，电阻和电容比较 'value'。

c)pin number / pin name / pin position 是否正确。

d)net hypergraph 连接关系是否正确。

e)线段是否在对应的 matched net 内画对。

f)Token消耗量。

本赛题不限制实现方法，参数队伍可应用机器学习算法、计算机视觉算法、多模态大模型等。

常见注意事项见附录四。

（二）提交说明

赛题需要提交两个部分，包括代码，技术报告。

1.代码：提交源代码或可执行文件。

1）本赛事提供 Python 及指定版本 PyTorch 的运行环境，该组合为赛事官方推荐的最佳技术栈。若参赛选手选用 Python 以外的编程语言，或使用不同版本的 Python / PyTorch，或采用其他深度学习框架（如 TensorFlow、Keras 等），须自行提供可运行的 Docker 镜像。

2.技术报告：以word格式提交队伍的作品介绍，包括：技术方案、实验结果以及实验总结。

五、评分标准

赛题最终分数FinalScore由两部分组成。第一部分由解决方案的准确度决定FinalScore_Accuracy，占100分。第二部分是由数据贡献产生的附加分FinalScore_Data，占10分，参赛队伍可在赛题提供的数据集PCBnet基础上自行收集数据，用于模型训练和算法优化。自行收集的数据集可提交到赛会提供的链接，正确和去重的案例数据会被计入附加分计算中。这些数据经确认后，也会补充到PCBnet开源数据集中。

FinalScore=FinalScore_Accuracy+FinalScore_Data

A. 解决方案准确度评分

约束条件

a)赛题提供基于qwen3.7-plus大模型API调用的测试环境。参赛队伍在开发过程中需考虑该测试环境因素，保证代码能够在赛会提供的测试平台上运行。

b)代码运行时间：完成所有案例的评测时间＜2小时。正式评测时会设置2小时的时间限制，评测开始后2小时截止，如果有未完成的测试例，该测试例将不会得分，总分也会受到相应的影响，参见下面的评分细则。

（一）评分总览

1.赛题打分案例在30~50个之间，公开（参赛队可见）和黑盒案例的比例是1：2。

2.赛题打分案例难度：按照中等40%，难60% 的比例设置。

3.最终准确度分数FinalScore_Accuracy由两部分组成，第一部分是案例评测准确度Score_Accuracy, 占总分值的95%，第二部分考虑Token消耗量Score_Token，占总分值的5%。假设赛题总打分案例为M例，某参赛队伍完成的测试例数量为m。

$$\text{FinalScore_Accuracy}=0.95\times\text{Score_Accuracy}+0.05\times\frac{m}{M}\times\text{Score_Token}$$

4.案例评测准确度分数Score_Accuracy为 0 到 100，由参赛队伍完成的所有案例准确度得分累加，再除以总打分案例数目。计算公式如下。

$$\text{Score_Accuracy}=\frac{1}{M}\sum_{1}^{m}Score_Accuracy_SingleCase$$

单一案例评测准确度分数Score_Accuracy_SingleCase为 0 到 100，由四个主要指标加权计算，计算公式如下。

$$\text{Score_Accuracy_SingleCase}=100\times(0.3\times\text{ComponentF1}+0.25\times\text{PinF1}+0.35\times\text{NetHypergraphF1}+0.1\times\text{NetL}$$

四个主要指标如下：

指标	权重	含义
ComponentF1	0.3	元件是否识别正确，包括 key、type、bbox 和必要文本
PinF1	0.25	pin number / pin name / pin position 是否正确
NetHypergraphF1	0.35	net 连接关系是否正确
NetLineF1	0.1	net 中的线段是否检测正确

5.参赛队伍 Token 总消耗量产生的分数 Score_Token 为 0 到 100，占总成绩的 5%，*Score_Token* 分数的计算采用赋分制，具体办法为：

假设有 N 支参赛队伍，把所有参赛队伍的 Token 消耗量从高到低排序，Token 消耗最多的队伍位列第一位，Token 消耗最少的队伍位列第 N 位;

把排序好的队伍分成 K 个区间（K 后期根据参赛队伍情况给出），每支参赛队伍根据位次被放置到相应的区间 n,其中 Token 消耗量最低队伍的会被放置到第 K 个区间，Token 消耗量最高的队伍会被放置到第 1 个区间;

如果一支队伍的位置处在第 n 个区间，则其赋分为:

$$Score_Token = n \times \frac{100}{K}$$

下表用一个例子说明，假设有 10 支参赛队伍 N=10，被分成 5 个赋分区间 K=5：

Token 使用量	排序位次	赋分区间	赋分分数
最多	1	1	20
依次递减	2		
依次递减	3	2	40
依次递减	4		
依次递减	5	3	60
依次递减	6		
依次递减	7	4	80
依次递减	8		
依次递减	9	5	100
最低	10		

(二) ComponentF1

Component 直接用 key 匹配，例如 GT 的 `U3` 只和 Pred 的 `U3` 比较。一个 component 需要同时满足以下条件才算正确：

a)key 匹配；

b)type 正确；

c)bbox 坐标误差在阈值内，默认阈值为‘20 px’；

若需要比较文本，则文本严格一致。

文本比较规则：

a)‘ic’：比较 ‘Name’；

b)‘resistor’、‘capacitor’：比较 ‘value’；

c)其他类别：不比较文本。

(三) PinF1

Pin 先按 component key 匹配，再按 pin number 匹配。一个 pin 需要满足：

a)Pin name 正确；

b)pin number 正确；

c)pin 坐标距离在阈值内，默认阈值为‘5 px’。pin 坐标和引脚连接方式见附录三的详细说明。

(四) NetHypergraphF1

每个 net 被视为一个 pin set。评测时根据 pred net 和 GT net 进行匈牙利匹配，之后计算每个 net 中的元件 pin 是否正确出现。

(五) NetLineF1

Line 只在 matched net pair 内比较。每条 `edge_i` 作为一条线段检测目标。若 Pred segment 与 GT segment 的两个端点距离足够接近，默认阈值为 `5 px`，则认为该线段检测正确。

(六) 注意事项

a) Component key 很重要，错误的 key 会影响 component、pin 和 net 的评分。

b) 字符串采用严格匹配，`10k`、`10K`、`10 k` 会被视为不同字符串。

c) Net key 本身不要求和 GT 一致，但`hyperGraph`中的 pin 连接关系需要正确。

d) 所有坐标单位均为图片像素。

e) 赛题会提供评测代码供参数队伍评测算法。完整的公式、边界情况和诊断指标说明会随代码一起提供。

f) 推荐目录结构：

```
submission/
  case_001/
    case_001.json
```

case_002/

case_002.json

B. 附加分 FinalScore_Data

1) 参赛队伍提交自行收集的数据可计入数据贡献额外附加分 FinalScore_Data，总分为 10 分。

2) 参赛队伍提交的正确案例数据量需满足最低要求方可获得附加分，具体最低数据量要求和分数计算方法会在赛题发布的《数据提交说明》文档中发布。

3) 参赛队伍提交的数据经验证和去重后会选取部分案例加入到评测数据集，每组不超过 10 例，赛题综合考虑数据的多样性和正确性。

4) 参赛队伍提交的数据经确认后会贡献到 PCBnet 开源数据集中，同时数据贡献人也会被加入到该数据集共同作者，参考文献[1]。

数据集名称：PCBnet (面向 PCB 原理图理解与网表构建的大规模数据集)

网站：https://github.com/ECNUACRush/PCBnet/

5) 数据提交方式与正确性评价标准会在赛题发布的《数据提交说明》文档中发布。该文档也会提供数据收集和数据标注的参考方法。

六、参考文献

[1] Zhen Huang， Yuhao Gao, Yuzhi Liu, Daian Cheng, Chengyuan Shao, Yucheng Chen, Yongjian Jia, Futing Zhang, Yichen Shi, Wenhao Wang, Zuyan He, Yangbo Wei, Zhanfei Chen, Jinlong Yan, Yu Zhang, Haoying Wu, Ting-Jung Lin, Lei He， PCBnet: A Dataset and Automatic Constructing of SPICE Netlists from Schematic Images，IEEE LAD 2026， July 29-31

[2] Yichen Shi，Zhoufu Tao，Yuhao Gao，Li Huang、Hongyang Wang，Zhiping Yu， Ting-Jung Lin， Lei He, AMSnet 2.0: A Large AMS Database with AI

Segmentation for Net Detection, 2025 IEEE International Conference on LLM-Aided Design (ICLAD)

[3] Taiting Lu, Kaiyuan Lin,Yuxin Tian, Yubo Wang, Sharique Khatri, Akshit Kartik, Yixi Wang, Amey Rane, Yida Wang, Yifan Yang, Yi-Chao Chen, Yicheng Jin, Mahanth Gowda, OmniSch: A Multimodal PCB Schematic Benchmark For Structured Diagram Visual Reasoning，arXiv:2604.00270

[4] Yuqi Jiang, Yupeng Hu, Jinyuan Deng, Xiaotian Qiu, Yucheng Cui, Xuyang He, Ruidong Li, Qi Sun, Cheng Xhou, Circuit-Think: A Multimodal Reasoning Framework for Automated Circuit-to-Netlist Translation with Trajectory-Guided Reinforcement Learning, Vol. 40 No.7:AAAI-26 Technical Tracks 7

[5] Jincheng Lou, Ruohan Xu, Jiapeng Li, Yunyin Pi, RunZhe Tao, Weijian Fan, Xiao Tan, Guojie Luo, Yibo Lin, DiagramNet: An End-to-End Recognition Framework and Dataset for Non-Standard System-Level Diagrams, arXiv:2605.01338

七、附录

附录一：输出 JSON 格式详解

本案例说明以树莓派 RP2350 Pico2 开发板原理图（图一）的解析结果为例，对输出 JSON 详细说明，包含目标 JSON 中 components 、 pins 、 nets 三个段落的格式要求、字段含义、典型示例及截图对照。

1.数据说明

输出目标 JSON 必须包含三个顶层字段：

```
{
  "components": {},
  "pins": {},
  "nets": {}
}
```

a)component：以元件 designator 为键，描述每个元件的类别、名称、取值和包围盒。

b)pins：以元件 designator 为键，下属 pin_ ，描述引脚名称和连接点坐标。

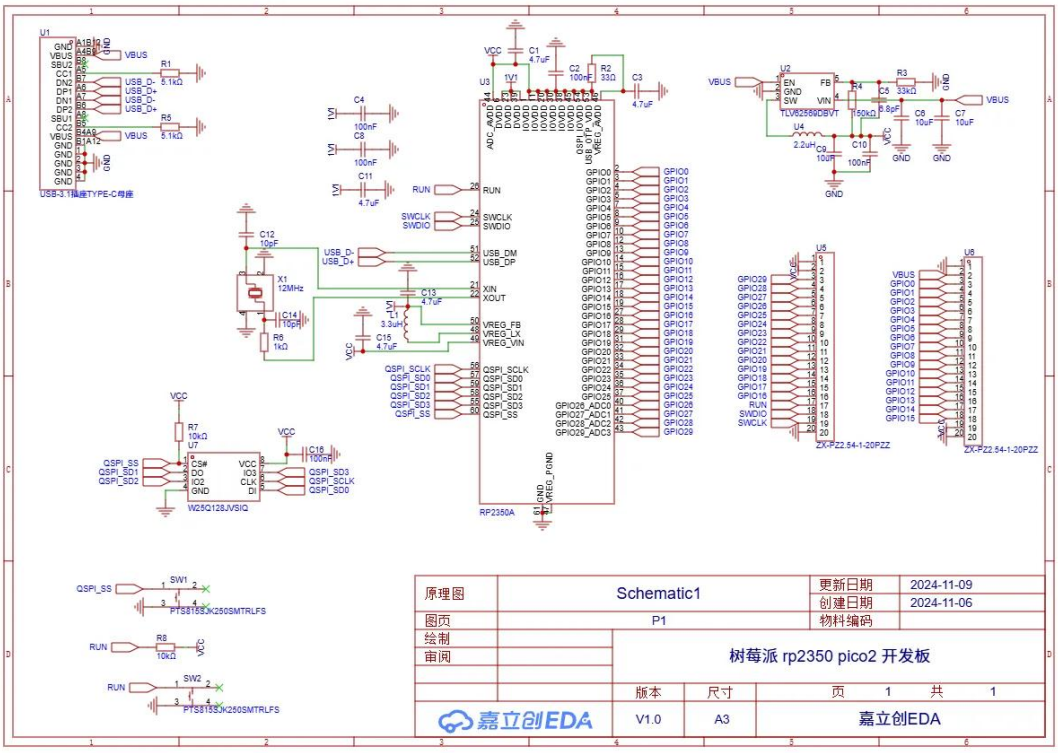
c)nets：以 net 名称为键，描述 hyperGraph（逻辑连接关系）和 edges（实际导线线段）。

2.坐标系说明

坐标系沿用原始 ParsedJson 的左下角坐标系：x 轴向右增大，y 轴向上增大。bbox 的格式为 [x1, y1, x2, y2] ，其中 (x1, y1) 为左下角坐标， (x2, y2) 为右上角坐标。

3.完整原理图预览

下图为本案例的完整原理图（同图一，为方便查看重新引用）。后续各小节将针对图中标注的局部区域，分别说明 boxic/box 类型元件、非 boxic/box 类型元件、引脚、不带飞线 net 以及带飞线 net 的 JSON 表示方法。



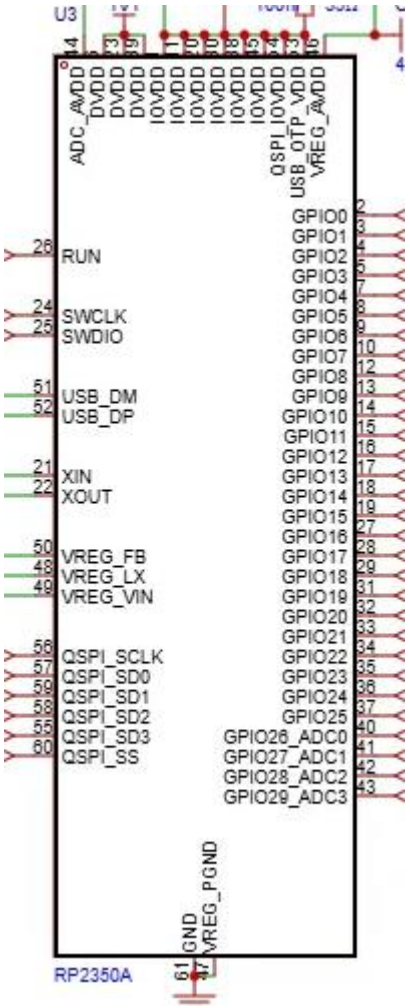
4.boxic / box 类型元件 JSON 示例

在本赛题评测体系中，"boxic" / "box" 类型通常指外形为矩形封装、内部包含多个引脚的集成电路（IC），在输出 JSON 中对应 type 字段为 "ic" 的元件。

本案例中的主控芯片 U3（RP2350A）即为典型代表。

截图示例

图二黑框内即为 U3（RP2350A），一个典型的 boxic/box 类型元件。该元件在原理图中表现为一个大型矩形框，四周排列着大量引脚。



图二：boxic/box 类型元件示例

JSON 示例

```
{
  "components": {
    "U3": {
      "type": "ic",
      "Name": "RP2350A",
      "value": null,
      "bbox": [540.0, 280.0, 690.0, 730.0]
    }
  }
}
```

字段说明

字段	类型	是否必需	说明
type	string	是	元件类别。"ic" 表示集成电路，对应赛事中的 boxic/box 类型
name	string/null	是	元件型号或名称。对于 ic 类型，该字段会进入 component text 评分
value	string/null	否	元件取值。IC 通常无 value，输出为 null
bbox	[x1,y1,x2,y2]	是	元件外框，使用左下角坐标系。x1 < x2，y1 < y2

补充说明：重复 designator 的统一编号示例

当同一原理图中出现多个同名元件时，转换器会统一编号为 _1、_2、_3 …，以保证输出 JSON 的键唯一。本案例中没有重复的 IC，但存在 4 个同名的 RUN 测试点/网络标签，它们在输出中被编号为 RUN_1、RUN_2、RUN_3、RUN_4。截图示例如图三，图三为 RUN_1 的局部截图（黑框标注）。RUN_2、RUN_3、RUN_4 分别位于图纸的右上、左下、左下区域，彼此没有直接画导线连接。



图三：多元件统一编号示例

JSON 示例


```
{
  "components": {
    "RUN_1": {
      "type": "vbus_in",
      "Name": "RUN",
      "value": null,
      "bbox": [490.0, 625.0, 520.0, 635.0]
    },
    "RUN_2": {
      "type": "vbus_in",
      "Name": "RUN",
      "value": null,
      "bbox": [865.0, 385.0, 895.0, 395.0]
    },
    "RUN_3": {
      "type": "vbus_in",
      "Name": "RUN",
      "value": null,
      "bbox": [130.0, 115.0, 160.0, 125.0]
    },
    "RUN_4": {
      "type": "vbus_in",
      "Name": "RUN",
      "value": null,
      "bbox": [150.0, 70.0, 180.0, 80.0]
    }
  }
}
```


字段说明

字段	类型	是否必需	说明
type	string	是	元件类别。`RUN` 在原始数据中被识别为网络标签/测试点，转换后类型为`"vbus_in"`
name	string/null	是	元件显示名称。4 个 `RUN_*` 的 `Name` 均为 `"RUN"
value	string/null	否	测试点无取值，输出为 `null`
bbox	[x1,y1,x2,y2]	是	每个测试点的包围盒，4 个 `RUN_*` 分布在原理图不同位置

注意事项

a) 统一编号规则适用于所有 designator，包括 ic 、 vbus_in 、 vcc 、 gnd 、 resistor 、 capacitor 等。仅出现一次的 designator 保持原名。

b) 编号顺序按照 component ID 的自然排序确定，因此第一个出现的 RUN 变为 RUN_1 ，第二个变为 RUN_2 ，以此类推。

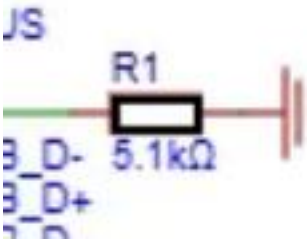
c) 如果 ic 类型出现重复，同样会按此规则编号，例如两个同名主控会变为 U3_1 、 U3_2 。

5.非 boxic / box 类型元件 JSON 示例

非 boxic/box 类型元件包括电阻、电容、电感、晶振、开关、连接器等。这类元件通常只有两个引脚或少量引脚，外形不是大型矩形 IC 封装。这里以电阻 R1 为例进行说明。

截图示例

图四为 R1 （5.1kΩ 电阻），一个典型的非 boxic/box 类型元件。图中可见其只有两个引脚，外形为电阻符号：



图四：非 boxic/box 类型元件示例

JSON 示例

```
{
  "components": {
    "R1": {
      "type": "resistor",
      "Name": "0603WAF5101T5E",
      "value": "5.1kΩ",
      "bbox": [185.0, 756.0, 205.0, 764.0]
    }
  }
}
```

字段说明

字段	类型	是否必需	说明
type	string	是	元件类别，例如 `"resistor"`、`"capacitor"`、`"inductor"`、`"crystal"`、`"switch"`、`"connector"` 等
name	string/null	是	元件型号或封装名。对于电阻/电容，该字段通常不进入评分，但保留在输出中
value	string/null	条件必需	对于电阻和电容，该字段会进入 component text 评分

bbox	[x1,y1,x2,y2]	是	元件外框

注意事项

a) 电阻和电容的 value 严格区分大小写，例如 "10k" 与 "10K" 会被判为不同。

b) bbox 不用于 component 匹配，只在 key matched component 上计算坐标误差。

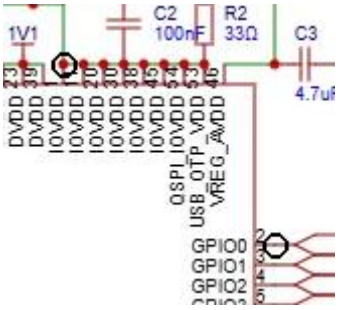
c) Component text 严格匹配：IC 比较 Name；电阻和电容比较 value；其他类别不比较 text。

6.引脚（pins）JSON 示例

pins 段落描述每个元件的引脚信息。第一层 key 必须对应 components 中的 designator；第二层 key 建议采用 pin_ 格式，其中来自原始数据中的 pinNumber 字段。如果 pinNumber 为空、⌀ 占位符或 OCR 噪声，则会自动编号为 pin_1 、pin_2 …。

截图示例

图五中黑圈标注了 U3 的 pin_1 （IOVDD，左侧）与 pin_2 （GPIO0，右侧）：



图五：引脚示例

JSON 示例

以下以 U3 的 pin_1 （IOVDD）和 pin_2 （GPIO0）为例：

```
{
  "pins": {
    "U3": {
      "pin_1": {
        "pinname": "IOVDD",
        "point": { "x": 595.0, "y": 740.0 }
      },
      "pin_2": {
        "pinname": "GPIO0",
        "point": { "x": 700.0, "y": 650.0 }
      }
    }
  }
}
```


字段说明

字段	类型	是否必需	说明
pinname	string	是	引脚名称，用于严格字符串评分
point.x	number	是	引脚端点 x 坐标
point.y	number	是	引脚端点 y 坐标

注意事项

a) Pin 匹配不是最近点匹配。评分时先要求 component key matched，再用 pin number 匹配。

b) pin_2 的 pin number 是 "2"， pin_A5 的 pin number 是 "A5" 。

c) 对于 pinNumber 为空或噪声的引脚，转换器会自动编号。例如 RUN_1 的唯一引脚会被输出为 pin_1 。

d) 若一个引脚对应多个编号（如 "5,6,7,8,9" ），转换器会拆成 pin_5 、 pin_6 等多个独立条目。

7. hyperGraph 相关 JSON 格式示例（不带飞线）

不带飞线的 net 指：hyperGraph 中所有引脚在原理图上已有实际导线直接相连，net 的 edges 段落描述了这些实际导线的几何线段。

本例选取 net $1N695 ，它连接 U1.A5 （USB Type-C 的 CC1 引脚）与 R1.1。

截图示例

图六中黑圈标注了 U1.A5 与 R1.1 ，两点之间由绿色导线直接相连。该 net

的 edges 中只有一条 edge_1，从 (100.0, 760.0) 到 (170.0, 760.0)，与图中导线完全对应：



图六：不带飞线的 net 示例

JSON 示例

```
{
  "nets": {
    "$1N695": {
      "hyperGraph": "(U1.A5,R1.1)",
      "edges": {
        "edge_1": [
          { "x": 100.0, "y": 760.0 },
          { "x": 170.0, "y": 760.0 }
        ]
      }
    }
  }
}
```

字段说明

字段	类型	是否必需	说明
hyperGraph	string	是	当前 net 逻辑连接的 pins，格式为 (component_id.pin_number,component_id.pin_number,...)
edges	object	是	当前 net 的实际导线线段集合；每个 edge_i 是一条两点线段

hyperGraph 规则:

(component_id.pin_number,component_id.pin_number,...)

例如 (U1.A5,R1.1) 会被解析为：U1.pin_A5， R1.pin_1

edges 规则:

- edges 是一个 object，key 建议使用 edge_1 、 edge_2 、…。
- 每个 edge_i 的 value 必须是长度为 2 的 point list： [start_point, end_point] 。
- 两个点可以相同，表示零长度线段或点状线段。
- 如果一条 wire 有多个折点，请拆成多个 edge_i 两点线段，不要写成 [p1, p2, p3]。 评分时是参考 edges 数量，数量要准确匹配，多余或缺少都会扣分。

注意事项

a) net key 本身不要求与 GT 完全一致，评分时 net 会通过 hyperGraph 内容做 Hungarian 匹配。

b) 每个 edge_i 必须恰好包含 2 个点，多于或少于 2 个点都会导致格式错误。

8. hyperGraph 相关 JSON 格式示例（带飞线）

什么是飞线？

飞线指：原理图上存在多个同名但物理位置分散的非 VCC/GND 测试点/网络标签元件，它们通过同一个 net 名称在逻辑上连接在一起。与这些同名元件直接相连的其它元件引脚，即使没有实际导线画在一起，也属于同一个 net。

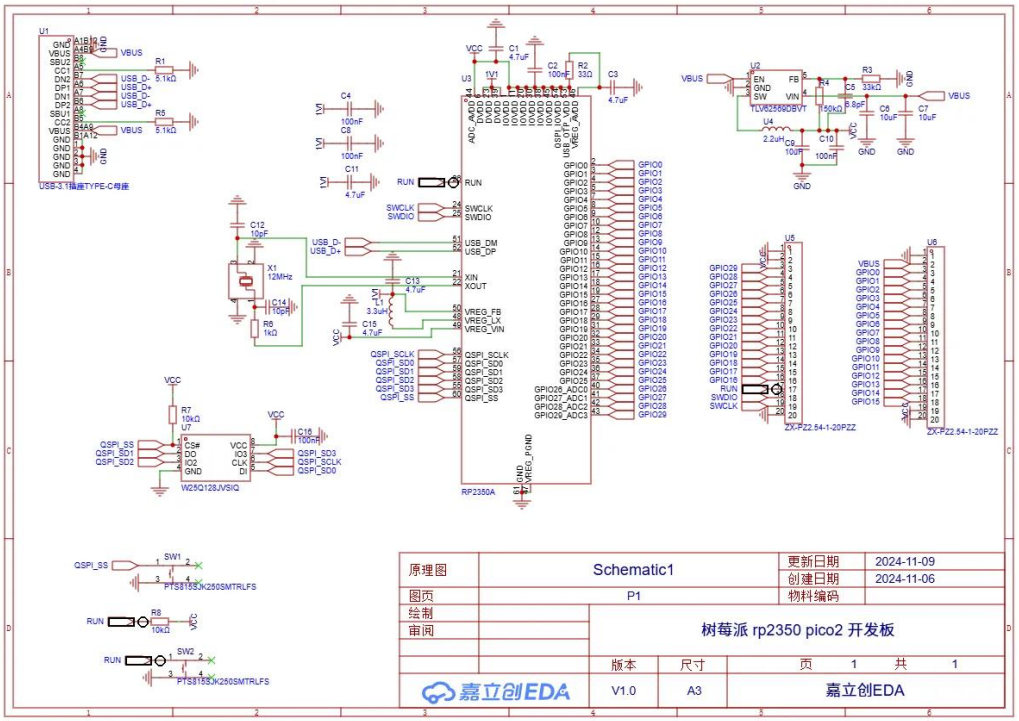
在本案例中，RUN_1 、RUN_2 、RUN_3 、RUN_4 是 4 个同名的测试点/网络标签，分别位于原理图的不同位置。它们共同构成了 RUN 网络：

- RUN_1 与 U3.26 直接相连；
- RUN_2 与 U5.17 直接相连；
- RUN_3 与 R8.1 直接相连；
- RUN_4 与 SW2.1 直接相连。

虽然 U3 、U5 、R8 、SW2 之间在原理图上没有画导线，但由于它们都连接到了同名 RUN 网络，因此逻辑上属于同一个 net，这就是飞线关系。

截图示例

图七为整张原理图，黑框标出了 U3 、U5 、R8 、SW2 等元件，黑框标出了 RUN net 涉及的各个引脚。这些点分布在图纸不同位置，通过 RUN 网络在逻辑上相连，即为带飞线的 net。



图七：带飞线的 net 示例

JSON 示例

```
{
  "nets": {
    "RUN": {
      "hyperGraph": "(U3.26,RUN_1.1,U5.17,RUN_2.1,R8.1,RUN_3.1,SW2.1,RUN_4.1)",
      "edges": {
        "edge_1": [
          { "x": 530.0, "y": 630.0 },
          { "x": 530.0, "y": 630.0 }
        ],
        "edge_2": [
          { "x": 905.0, "y": 390.0 },
          { "x": 905.0, "y": 390.0 }
        ],
        "edge_3": [
          { "x": 170.0, "y": 120.0 },
          { "x": 170.0, "y": 120.0 }
        ],
        "edge_4": [
          { "x": 190.0, "y": 75.0 },
          { "x": 190.0, "y": 75.0 }
        ]
      }
    }
  }
}
```

字段说明

字段	类型	是否必需	说明
hyperGraph	string	是	当前 net 逻辑连接的 pins，包含所有通过飞线关联的引脚
edges	object	是	当前 net 的实际导线线段集合。飞线 net 的 edges 通常为各连接点处的零长度线段

飞线解析

hyperGraph 字符串

(U3.26,RUN_1.1,U5.17,RUN_2.1,R8.1,RUN_3.1,SW2.1,RUN_4.1) 会被解析为以下 8 个引脚引用：

U3.pin_26

RUN_1.pin_1

U5.pin_17

RUN_2.pin_1

R8.pin_1

RUN_3.pin_1

SW2.pin_1

RUN_4.pin_1

飞线特点总结

a) 同名分散元件：飞线 net 的核心是多个同名网络标签/测试点，如 RUN_1 、RUN_2 、 RUN_3 、 RUN_4 。

b) 逻辑连接而非物理连接：这些同名元件在图纸上没有导线相连，但共享同一个 net 名称。

c) edges 为点状线段：由于飞线没有实际 wire，每个连接点在 edges 中表现为一条零长度线段（起点与终点相同）。

d) hyperGraph 包含所有相关引脚：不仅包含同名测试点本身，还包含与它们直接相连的所有元件引脚。

注意事项

a) VCC/GND 等电源/地网络虽然也有多个同名元件，但通常被视为普通电源网络，不按飞线处理。飞线特指非 VCC/GND 的同名网络标签/测试点。

b) 如果同名测试点只有一个，则该 net 为普通 net，不形成飞线。

c) 飞线 net 的 hyperGraph 必须包含所有逻辑相连的引脚，遗漏任何一方都会导致连接关系错误。

附录二、完整最小输出示例

结合附录一的说明，以下为本案例的一个简化完整输出示例：

```
{
  "components": {
    "U3": {
      "type": "ic",
      "Name": "RP2350A",
      "value": null,
      "bbox": [540.0, 280.0, 690.0, 730.0]
    },
    "RUN_1": {
      "type": "vbus_in",
      "Name": "RUN",
      "value": null,
      "bbox": [490.0, 625.0, 520.0, 635.0]
    },
    "R1": {
      "type": "resistor",
      "Name": "0603WAF5101T5E",
      "value": "5.1kΩ",
      "bbox": [185.0, 756.0, 205.0, 764.0]
    }
  },
  "pins": {
    "U3": {
      "pin_1": { "pinname": "IOVDD", "point": { "x": 595.0, "y": 740.0 } },
      "pin_2": { "pinname": "GPIO0", "point": { "x": 700.0, "y": 650.0 } },
      "pin_26": { "pinname": "RUN", "point": { "x": 530.0, "y": 630.0 } }
    },
    "RUN_1": {
      "pin_1": { "pinname": "RUN", "point": { "x": 530.0, "y": 630.0 } }
    },
    "R1": {
      "pin_1": { "pinname": "1", "point": { "x": 175.0, "y": 760.0 } }
    }
  },
  "nets": {
    "$1N695": {
      "hyperGraph": "(U1.A5,R1.1)",
      "edges": {
        "edge_1": [
          { "x": 100.0, "y": 760.0 },
          { "x": 170.0, "y": 760.0 }
        ]
      }
    },
    "RUN": {
      "hyperGraph": "(U3.26,RUN_1.1,U5.17,RUN_2.1,R8.1,RUN_3.1,SW2.1,RUN_4.1)",
      "edges": {
        "edge_1": [
          { "x": 530.0, "y": 630.0 },
          { "x": 530.0, "y": 630.0 }
        ]
      }
    }
  }
}
```


附录三、引脚坐标和引脚连接方式说明

本说明以图一中的原理图为例，说明目标JSON中两种常见引脚连接方式的判定与表示方法：

a) 绿色导线连接红色引脚：导线端点与元件引脚在同一位置，绿色导线与红色引脚的交界点即为引脚坐标。

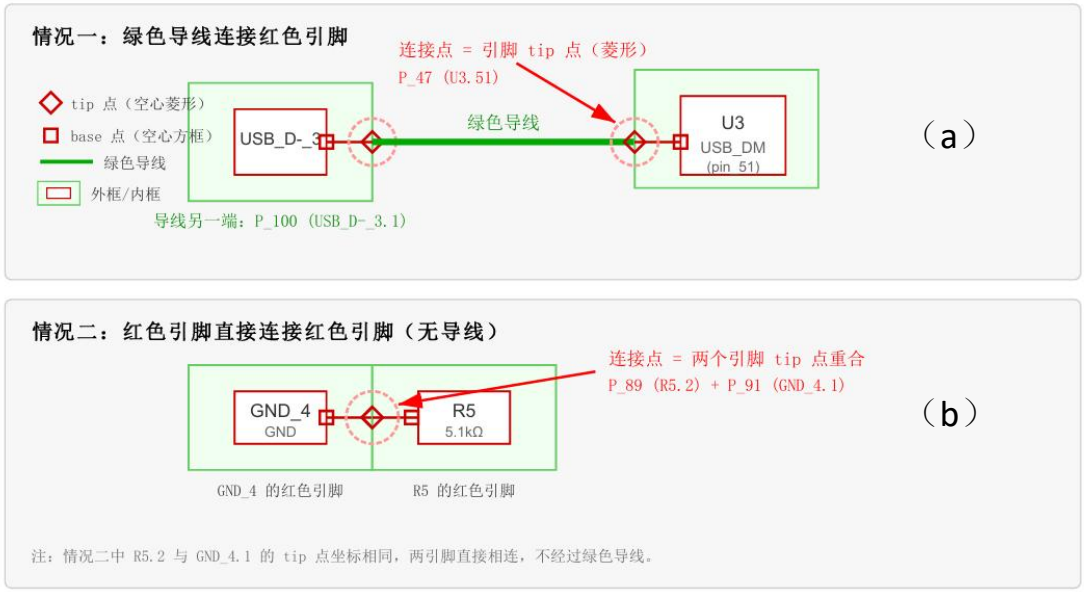
b) 红色引脚直接连接红色引脚：两个元件的引脚端点重合，直接在同一坐标处电气相连。

c) 坐标系沿用原始 ParsedJson 的左下角坐标系：x 轴向右增大，y 轴向上增大。 point的格式为 { "x": x, "y": y } 。

1.示例说明

图八用示意的方式展示了两种连接方式的核心要素。每个元件均包含外侧框（outer bbox）和内侧框（body bbox）：引脚tip点位于外侧框边缘，base点连至内侧框。

图八（a）为“绿色导线连接红色引脚”，绿色导线连接两个元件外侧框上的 tip 点。图八（b）为“红色引脚直接连接红色引脚”，两个元件外侧框相接，两个引脚的 tip 点重合，不经过绿色导线。



图八：引脚连接方式示意图

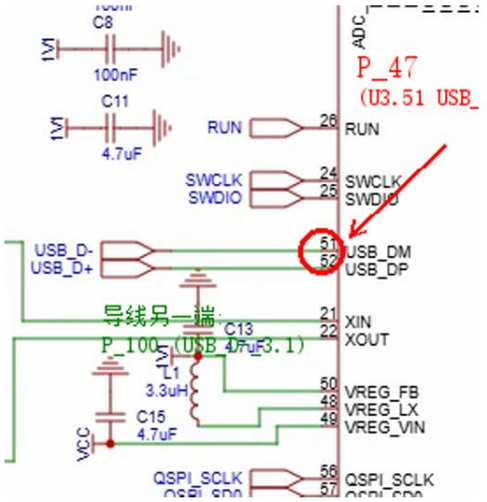
2.绿色导线连接红色引脚

场景说明

当原理图中存在一段实际绘制的导线（wire）时，导线通常显示为绿色线段。导线与元件引脚（红色端点）的交界位置即为该引脚的连接点坐标。输出 JSON 中，该引脚的 point 应记录这个交界点。

截图示例

图九为 U3 （RP2350A）左侧 USB 区域。红色圆圈标注了 U3 的第 51 号引脚 USB_DM（即 P_47 ），红色箭头指向该引脚。绿色水平导线从该引脚向左延伸，另一端连接到网络标签 USB_D-_3 的引脚 P_100（即 USB_D-_3.1 ）。



图九：引脚连接示意图

JSON 示例


```
{
  "pins": {
    "U3": {
      "pin_51": {
        "pinname": "USB_DM",
        "point": { "x": 530.0, "y": 560.0 }
      }
    },
    "USB_D-_3": {
      "pin_1": {
        "pinname": "P_100",
        "point": { "x": 445.0, "y": 560.0 }
      }
    }
  },
  "nets": {
    "USB_D-": {
      "hyperGraph": "(U3.51,USB_D-_3.1,U1.A7,USB_D-_2.1,U1.B7,USB_D-_1.1)",
      "edges": {
        "edge_1": [
          { "x": 530.0, "y": 560.0 },
          { "x": 445.0, "y": 560.0 }
        ],
        "edge_2": [
          { "x": 100.0, "y": 730.0 },
          { "x": 100.0, "y": 730.0 }
        ],
        "edge_3": [
          { "x": 100.0, "y": 750.0 },
          { "x": 100.0, "y": 750.0 }
        ]
      }
    }
  }
}
```


3.红色引脚直接连接红色引脚

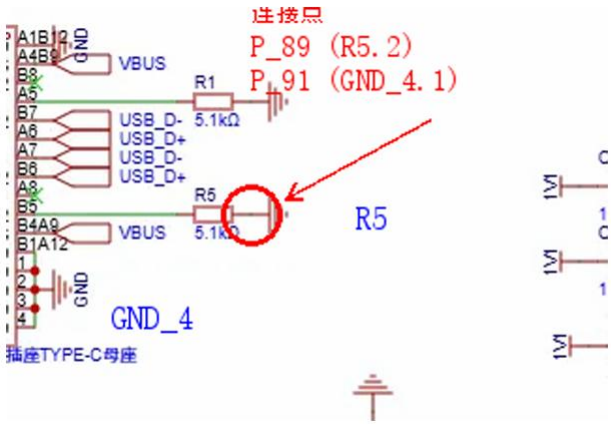
场景说明

当两个元件的引脚端点直接重合时，原理图上不会绘制独立的绿色导线，而是两个红色引脚端点在同一位置直接电气相连。此时需要在连接点处画圈标注，并用箭头明确指出该连接点；同时应把两个元件都截入图中，以便说明是哪两个引脚相连。

截图示例

图十为 R5（5.1kΩ 电阻）与 GND_4（GND 网络标签/接地符号）的局部区

域。红色圆圈标注了两个红色引脚的直接连接点，红色箭头指向该点。连接点同时是 R5 的第 2 号引脚 P_89 和 GND_4 的第 1 号引脚 P_91 的坐标位置。



图十：引脚连接示意图

JSON 示例


```
{
  "pins": {
    "R5": {
      "pin_1": {
        "pinname": "1",
        "point": { "x": 175.0, "y": 700.0 }
      },
      "pin_2": {
        "pinname": "2",
        "point": { "x": 215.0, "y": 700.0 }
      }
    },
    "GND_4": {
      "pin_1": {
        "pinname": "P_91",
        "point": { "x": 215.0, "y": 700.0 }
    "GND": {
      "hyperGraph": "
(U3.47,U3.61,GND_5.1,U1.1,U1.2,GND_1.1,U1.3,U1.4,U1.B1A12,U1.A1B12,GND_2.1,R1.2,GND
_3.1,R5.2,GND_4.1,C13.2,GND_7.1,C15.2,GND_6.1,C3.2,GND_8.1,X1.2,GND_9.1,X1.4,GND_10
.1,C12.2,GND_12.1,C14.1,GND_11.1,C4.2,GND_13.1,C8.2,GND_14.1,C11.2,GND_15.1,C1.2,GN
D_16.1,C2.2,GND_17.1,U7.4,GND_18.1,C16.2,GND_19.1,U2.2,GND_20.1,C6.2,GND_21.1,C9.2,
GND_22.1,C10.2,R3.2,GND_23.1,C7.2,GND_24.1,U5.1,GND_25.1,U5.20,GND_27.1,U6.1,GND_26
.1,U6.20,GND_28.1,SW1.3,GND_29.1,SW2.3,GND_30.1)",
      "edges": {
        "edge_8": [
          { "x": 215.0, "y": 700.0 },
          { "x": 215.0, "y": 700.0 }
        ]
      }
    }
  }
}
```

附录四、常见注意事项

1. Component key 非常重要。`U3` 写成 `U8` 会导致该 component、其 pins、相关 nets 都难以得分。
2. Pin key 要稳定使用 `pin_<pin_number>`，因为 pin matching 直接使用 pin number。
3. `hyperGraph` 中的 pin 写法应和 pin key 对齐，例如 `U1.2` 对应 `pins.U1.pin_2`。
4. Net key 本身不要求和 GT 一致，但 net 的 `hyperGraph` pin set 要正确。
5. Line geometry 必须放在正确 net 的 `edges` 下；画到错误 net 里不会因为几何位置接近而得分。
6. 字符串采用严格匹配，`10k`、`10K`、`10 k` 是不同字符串。
7. 坐标单位为图片像素，默认 pin 和 line 阈值都是 `5 px`。

*本赛题指南未尽问题，见赛题Q&A文件